generate_pdfs.py

```
import os
import json
from fpdf import FPDF

# Configuration
SOURCE_DIR = "/Users/adityagowari/Programs/Machine_Learning_Playground"
OUTPUT_DIR = os.path.join(SOURCE_DIR, "all_code_pdfs")
EXTENSIONS_TO_CONVERT = {'.py', '.ipynb', '.md', '.txt', '.cpp', '.cu', '.h', '.c'}
EXCLUDE_DIRS = {'.git', '.venv', '.conda', '__pycache__', '.ipynb_checkpoints', 'all_code_pdfs'}

class CodeToPDF(FPDF):
    def footer(self):
        self.set_y(-15)
        self.set_font('Arial', 'I', 8)
        self.cell(0, 10, f'Page {self.page_no()}', 0, 0, 'C')

def convert_text_to_pdf(content, output_path, title):
    pdf = CodeToPDF()
    pdf.add_page()
    pdf.set_font("Courier", size=10)

    # Title
    pdf.set_font("Arial", 'B', 12)
    pdf.cell(0, 10, title, 0, 1, 'C')
    pdf.ln(5)

    pdf.set_font("Courier", size=8)
    # Split content by lines and handle long lines
    lines = content.split('\n')
    for line in lines:
        # Simple line wrapping: fpdf's multi_cell is better but we might want more control
        # or just use multi_cell for each line to allow wrapping.
        # Encode to latin-1 for FPDF 1.7.2 compatibility, replacing unknown chars
        safe_line = line.encode('latin-1', 'replace').decode('latin-1')
        pdf.multi_cell(0, 5, safe_line)

    try:
        pdf.output(output_path)
        return True
    except Exception as e:
        print(f"Error saving PDF {output_path}: {e}")
        return False

def extract_ipynb_code(filepath):
    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            nb = json.load(f)

        content = []
        for cell in nb.get('cells', []):
```

```
            cell_type = cell.get('cell_type')
            source = "".join(cell.get('source', []))

            if cell_type == 'code':
                content.append(f"### CODE CELL ###\n{source}\n")
            elif cell_type == 'markdown':
                content.append(f"### MARKDOWN CELL ###\n{source}\n")

        return "\n".join(content)
    except Exception as e:
        return f"Error parsing notebook: {e}"

def main():
    if not os.path.exists(OUTPUT_DIR):
        os.makedirs(OUTPUT_DIR)

    files_processed = 0
    for root, dirs, files in os.walk(SOURCE_DIR):
        # Skip excluded directories
        dirs[:] = [d for d in dirs if d not in EXCLUDE_DIRS]

        for file in files:
            name, ext = os.path.splitext(file)
            if ext.lower() in EXTENSIONS_TO_CONVERT:
                full_path = os.path.join(root, file)
                rel_path = os.path.relpath(full_path, SOURCE_DIR)

                # Create a safe filename for the PDF
                safe_name = rel_path.replace(os.sep, '_') + ".pdf"
                output_path = os.path.join(OUTPUT_DIR, safe_name)

                print(f"Converting: {rel_path}")

                if ext.lower() == '.ipynb':
                    content = extract_ipynb_code(full_path)
                else:
                    try:
                        with open(full_path, 'r', encoding='utf-8', errors='replace') as f:
                            content = f.read()
                    except Exception as e:
                        content = f"Error reading file: {e}"

                if convert_text_to_pdf(content, output_path, rel_path):
                    files_processed += 1

    print(f"\nSuccessfully processed {files_processed} files.")
    print(f"PDFs are saved in: {OUTPUT_DIR}")

if __name__ == "__main__":
    main()
```